



Data Structures

CPT102 – Lecture 8b
Erick Purwanto



Xi'an Jiaotong-Liverpool University

西交利物浦大學

CPT102 Data Structures

Lecture 8b

Priority Queue and Binary Heap

Welcome !

- Welcome to Lecture 8b !
- In this lecture we are going to
 - learn about priority queues and their operations
 - implement priority queues using binary heaps

Priority Queue

- Priority queue is an abstract data structure that supports the operations :
 - add an item: `add(T item)`
 - delete smallest item: `T delMin()`
- For example:

```
add("A")  
add("B")  
add("C")  
delMin()    →    "A"  
add("D")  
delMin()    →    "B"
```

Application Example

- Problem : Find the *largest* M items in a stream InputStream of N items
 - Fraud detection : isolate transactions
 - File maintenance : find biggest files or directories
- Constraint : Not enough memory to store N items

N huge, M large



```
MinPQ<Transaction> pq = new MinPQ<Transaction>();
while (InputStream.hasNextLine()) {
    String line = InputStream.readLine();
    Transaction item = new Transaction(line);
    pq.add(item);
    if (pq.size() > M)
        pq.delMin();
}
```

Applications of Priority Queues

- Applications of Priority Queues:
 - Customers in a line, Colliding particles simulations
 - Reducing roundoff error in numerical computation
 - Huffman codes
 - Dijkstra's algorithm, Prim's algorithm
 - Sum of powers
 - A* search
 - Maintain largest M values in a sequence
 - Load balancing, Interrupt handling
 - Bin packing, Scheduling
 - Bayesian spam filter

MinPQ Interface

- Minimum Priority Queue supports fast removal of the smallest item in the data structure and fast item addition
- To have a notion of smallest, either we implement the item to be comparable, or explicitly specify the priority of an item with a comparable value, for example an integer
- Interface MinPQ with comparable items:

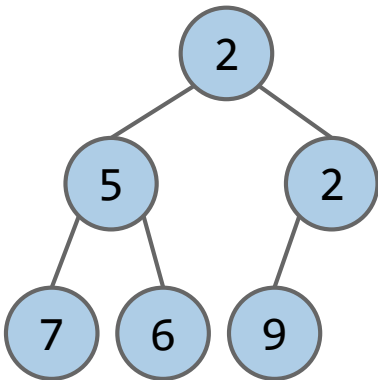
```
public interface MinPQ<T extends Comparable<T>> {  
    void add(T item);  
    T getMin();  
    T delMin();  
    boolean isEmpty();  
    int size();  
}
```

Implementing MinPQ

- Implemented naively using an **array**, we can achieve constant-time item addition but linear-time smallest item removal
 - On the other hand, using an **ordered array**, constant-time removal but linear-time addition
- Our goal is then to have an implementation that can achieve fast running-time for *both* operations
 - Binary heap is suitable for this

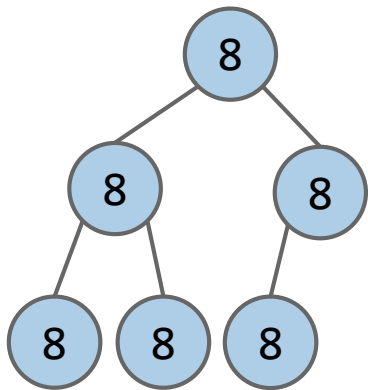
Binary Heaps

- A **binary heap** is a *binary tree* that is complete and satisfies the heap property :
 - **Complete**: balanced binary tree, could be left-aligned at the bottom level
 - **Heap property**: parent's item/priority is smaller or equal to the children's
- For example :

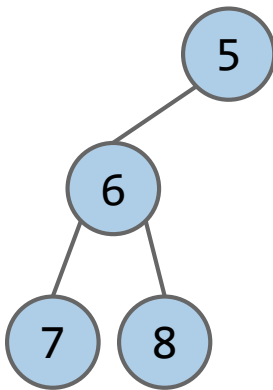


Binary Heaps ?

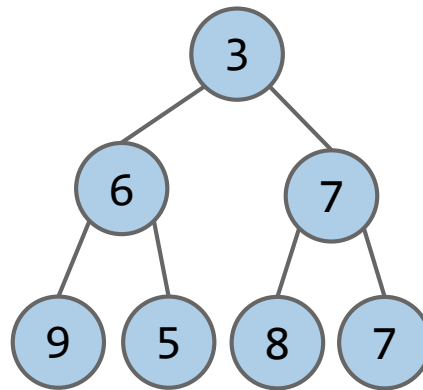
- Which of the following are binary heaps ?



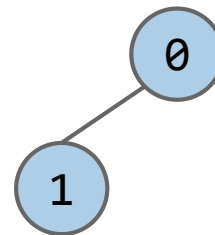
(i)



(ii)



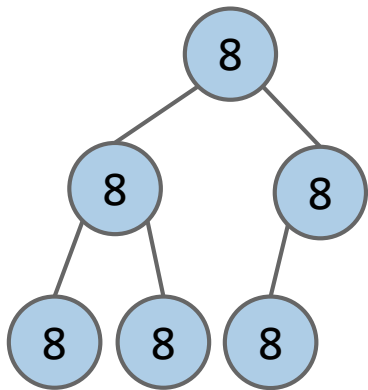
(iii)



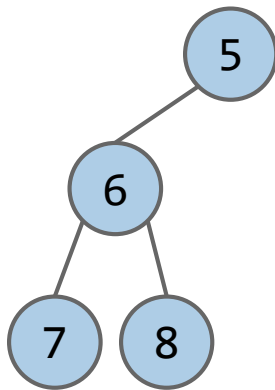
(iv)

Binary Heaps ?

- Which of the following are binary heaps ?

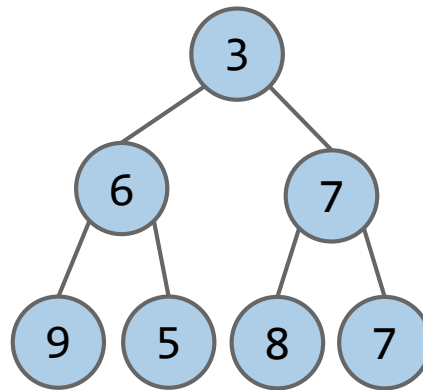


(i)



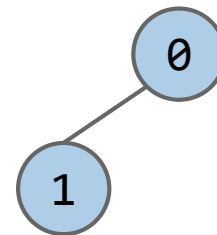
(ii)

incomplete



(iii)

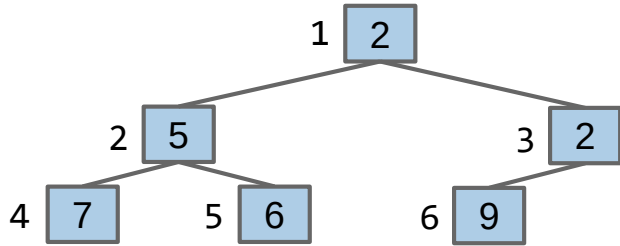
lacking min-heap
property



(iv)

Binary Heap Representation

- If we represent the binary heap by an array or an ArrayList, set the indices to start at 1, and order the nodes containing the item and the priority by the heap-property :
 - there is no explicit links needed between parent and the children
 - we can use the indices to access an item's parent or children
 - parent of an item at index k is $k/2$
 - children of an item at index k is $2*k$ and $2*k + 1$
- For example:



T[] heap

-	2	5	2	7	6	9	
0	1	2	3	4	5	6	7

size = 6
capacity = 7
heap.length = 8

Comparable Items and Swap

- Since the items are comparable, we can create a helper method to decide whether an item at index i is *greater than* the item at index j

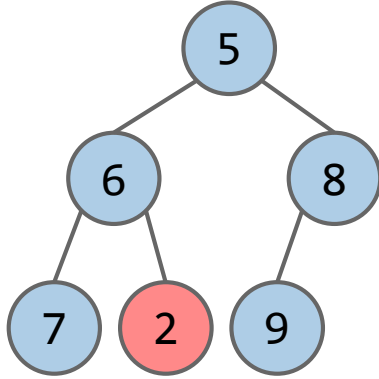
```
private boolean greater(int i, int j) {  
    return ((Comparable<T>) heap[i]).compareTo(heap[j]) > 0;  
}
```

- In the next slides, we will implement operations that swap (the reference to) the items

```
private void swap(int i, int j) {  
    T temp = heap[i];  
    heap[i] = heap[j];  
    heap[j] = temp;  
}
```

Restoring Heap Property : Swim

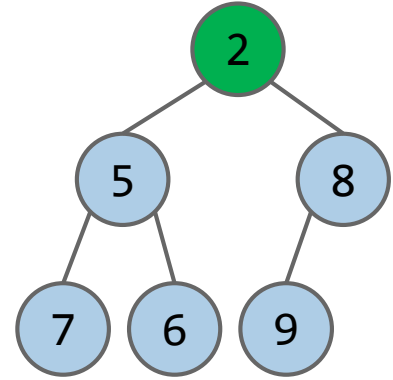
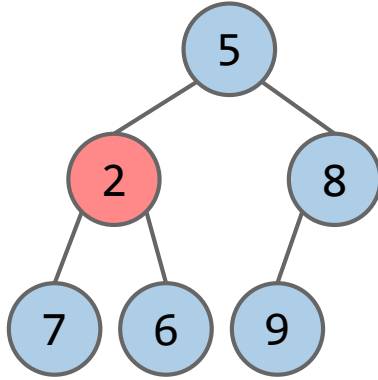
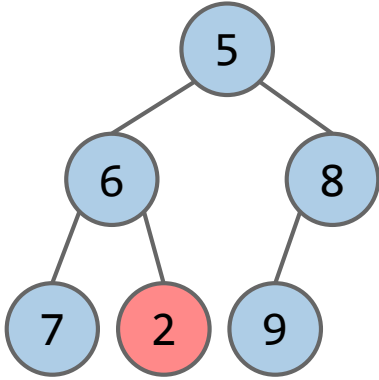
- What should we do if a child becomes smaller than its parent ?



swap child with the parent,
until heap property is restored

Restoring Heap Property : Swim

- What should we do if a child becomes smaller than its parent ?

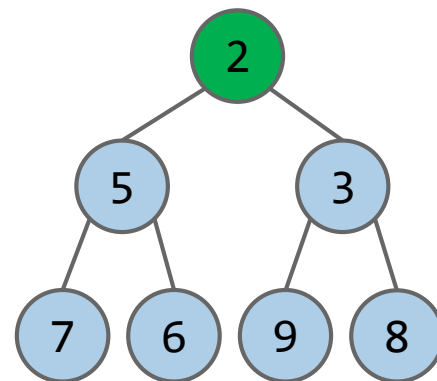
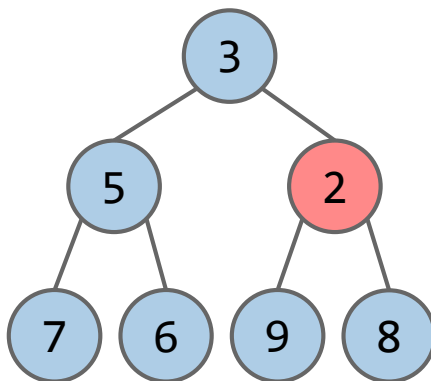
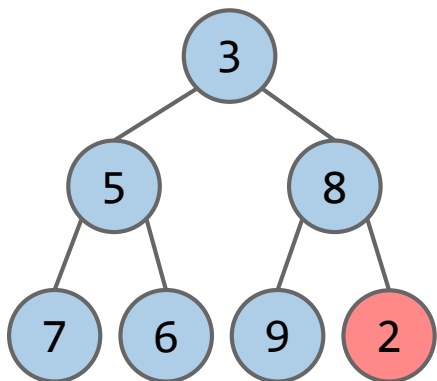


```
private void swim(int k) {  
    while (k > 1 && greater(k/2, k)) {  
        swap(k, k/2);  
        k = k/2;  
    }  
}
```

swap child with the parent,
until heap property is restored

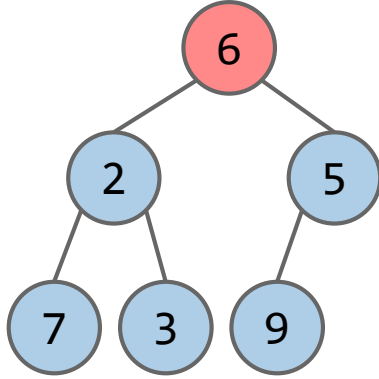
Item Addition

- Item addition:
 - add the new item at the end
 - then swim it : swap it with its parent repeatedly until the heap-property is restored



Restoring Heap Property : Sink

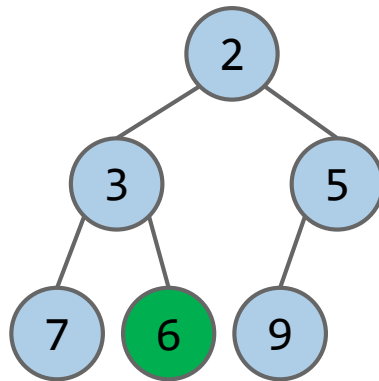
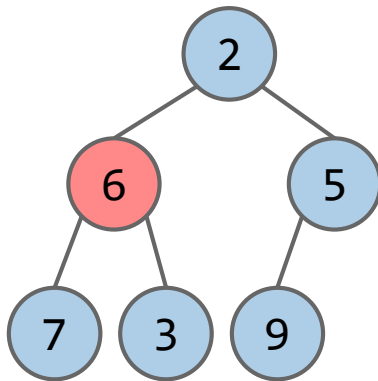
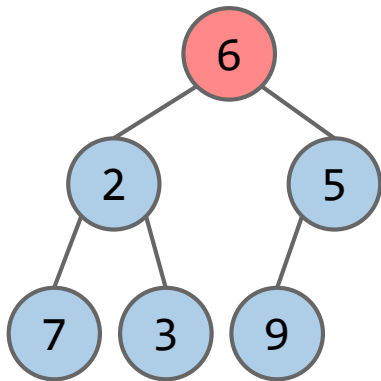
- What should we do if a parent becomes larger than one child / both of its children ?



swap parent with the **smaller** child,
until heap property is restored

Restoring Heap Property : Sink

- What should we do if a parent becomes larger than one child / both of its children ?

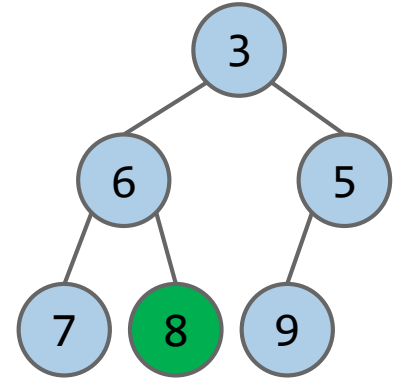
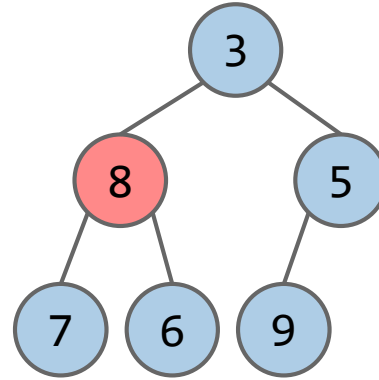
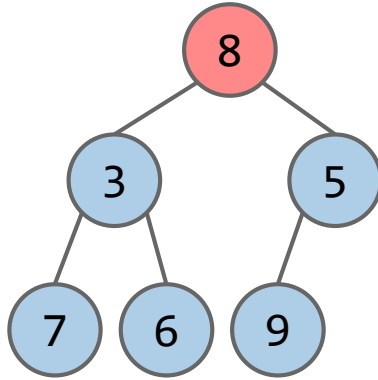
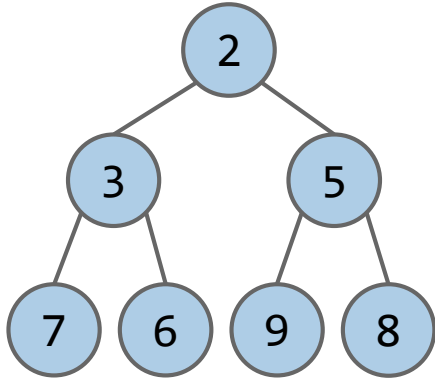


```
private void sink(int k) {  
    while (2*k <= size) {  
        int j = 2*k;  
        if (j < size && greater(j, j+1)) j++;  
        if (!greater(k, j)) break;  
        swap(k, j);  
        k = j;  
    }  
}
```

swap parent with the **smaller** child,
until heap property is restored

Minimum Item Removal

- Delete min:
 - replace the minimum item at the root with the last item
 - then sink it : swap it with the smaller children repeatedly until the heap-property is restored



Running Analysis of Binary Heap

How many comparisons?

- Item addition: $O(\text{height of a binary tree}) = O(\log N)$
- Minimum item removal: $O(2 \times \text{height of a binary tree}) = O(\log N)$

Representation	add	delMin
array	$O(1)$	$O(N)$
ordered array	$O(N)$	$O(1)$
binary heap	$O(\log N)$	$O(\log N)$

Thank you for your attention !

- In this lecture, you have learned:
 - about the priority queue and binary heap data structures
- Please continue to work on **Exercise 8**